

React Component Using a REST Endpoint

Introduction

The aim of this task was to build a simple React component that connects to a REST API and displays guest information. I used an AI coding assistant to help create the component and implemented loading and error handling.

AI Coding Assistant Prompt

Create a simple React component that fetches guest data from a REST API using `fetch()`. Display the guest name and email. Include loading and error states. Use `useState` and `useEffect` and keep the code simple and easy to understand.

React Component

```
import { useEffect, useState } from "react";

function GuestList() {
  const [guests, setGuests] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState("");

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((response) => {
        if (!response.ok) {
          throw new Error("Failed to fetch guest data");
        }
        return response.json();
      })
      .then((data) => {
        setGuests(data);
        setLoading(false);
      })
      .catch((err) => {
```

```

        setError(err.message);
        setLoading(false);
    });
}, []));

if (loading) {
    return <p>Loading guests...</p>;
}

if (error) {
    return <p>Error: {error}</p>;
}

return (
    <div>
        <h2>Guest List</h2>

        {guests.map((guest) => (
            <div key={guest.id}>
                <h3>{guest.name}</h3>
                <p>{guest.email}</p>
            </div>
        ))}
    </div>
);
}

export default GuestList;

```

How the Component Works

1. useState

useState is used to manage the component's data.

- `guests` stores the guest information.

- loading keeps track of whether the API is still loading.
- error stores an error message if something goes wrong.

2. useEffect

useEffect is used to call the REST API when the component loads.

```
useEffect(() => {  
  // API call  
}, []);
```

The empty [] means the API call runs once when the component is loaded.

3. REST API

The fetch() function sends a request to the REST endpoint and receives guest data in JSON format.

4. Loading State

While the API is processing, the user sees:

Loading guests...

5. Error State

If the API request fails, an error message is displayed instead of the guest list.

6. Displaying Data

After successfully receiving the data, the component uses .map() to display each guest's name and email.

Overall Flow

React Component → API Request → Loading State → API Response → Store Data → Display Guest List

If the request fails:

React Component → API Request → Error → Display Error Message

Key Learning

Through this task, I learned how to connect a React component with a REST API using `fetch()`. I also understood how `useState` manages changing data and how `useEffect` is used to perform the API call when the component loads. Using an AI coding assistant also helped me create the basic component faster, while I learned the importance of reviewing and understanding the generated code.